

SAFESCALE

A hybrid multi-agent and multi-objective system for SAFe PI Planning

NSGA-II, Critical Chain and conditional HITL optimization

Gabriel Leulmi

Master's Thesis, 2025-2026

May 2026

Abstract

Planning a *Program Increment* (PI, a delivery cycle of eight to twelve weeks) within the Scaled Agile Framework [22] is a **resource-constrained scheduling problem**: one must decide what to do, when, and by which team, while respecting the available skills, the dependencies and the business deadlines. This type of problem (called RCPSP in operations research [21, 20]) is **non-polynomial**: the number of possible schedules explodes so fast with project size that no computer can test them all. Planning therefore relies today on human experience. The *Release Train Engineer* (RTE, facilitator of the delivery train) runs the ritual, but several other SAFe roles are involved: *Scrum Master*, *Product Owner* (PO), *Chief Product Owner* (CPO), *Engineering Delivery Managers* and the *Development Teams* that provide estimates and *refinement*. Market tools serve to capture and visualize these exchanges, not to explore several candidate scenarios or to evaluate them against multiple criteria. This thesis presents **SAFESCALE**, a decision-support prototype that assembles three elements. First, an optimization by **multi-objective genetic algorithm** (NSGA-II [24, 8], which evolves a population of candidate schedules over several generations), extended to six objectives including a **WSJF-weighted tardiness** (the delay of each feature in days, multiplied by its business value), inspired by a multi-project industrial case [15], with a **local search** that refines the best schedules (“memetic” approach [4]) and starting schedules built by domain heuristics (inspired by INSGA-II [1, 2]). Second, a multi-agent architecture *orchestrated* in five steps, with a human in the loop in a **conditional** manner (consulted only when a problem is detected) and an explicit trace of decisions. Finally, a deterministic risk model inspired by Goldratt’s *Critical Chain* and Kingman’s law [25], which inserts time margins (*buffers*) after the riskiest *stories*. The system accounts for the effort-duration / calendar-duration ratio (a story point is rarely executed in as many days as it is worth), team specialization (*one team owns one feature* [22]) and capacity bottlenecks per skill. It is instantiated on a reproducible synthetic dataset (one ART, nine teams, four three-week sprints, ≈ 135 *stories*) whose dimensions are inspired by an interview with a Product Owner.

Keywords. SAFe ; PI Planning ; RCPSP ; memetic NSGA-II ; orchestrated multi-agent system ; conditional HITL ; *Critical Chain* ; Kingman’s law ; WSJF-weighted tardiness.

1 Introduction

1.1 Context

The *Scaled Agile Framework* (SAFe) defines the planning of an *Agile Release Train* (ART, see appendix A) gathering 5 to 15 teams and 50 to 125 people, organized around a central ceremony, the **PI Planning**, over one to two days to plan the following eight to twelve weeks [22]. This ritual mobilizes several roles whose responsibilities are precisely distributed:

- the *Release Train Engineer* (RTE) runs the ritual and arbitrates dependencies between teams;
- the *Chief Product Owner* (CPO) prioritizes the *features* in his scope and distributes them across teams;
- each *Product Owner* (PO) carries the content of his team and presents its proposals;
- the *Scrum Master* (SM) facilitates the ceremony on the team side;
- the *Engineering Delivery Managers* (EDM) and the other stakeholders accept or refuse the proposed commitments;

- the *Development Teams* produce the estimates and the *refinement* that feed all decisions.

None of these roles today has a tool to explore several candidate scenarios and evaluate them quantitatively; trade-offs are made by eye, on the basis of experience.

1.2 Motivation

Formally, what the RTE must do falls under the *Multi-Skill Resource-Constrained Multi-Project Scheduling Problem* (MS-RCMPSP), an extension of the RCPSP [21] proven NP-hard as soon as 60 activities [20]. “Optimizing” here means three concrete things: (i) generating several candidate PI scenarios instead of a single one, (ii) evaluating them simultaneously against several criteria, and (iii) recommending a compromise while keeping the final decision with the human. This complexity is not addressed by existing tools, which remain focused on capturing and visualizing SAFe artifacts. Optimization research has nonetheless worked extensively on neighboring problems: hybrid variants of NSGA-II [4, 5] and NSGA-III [10, 12] give good results on instances of RCPSP and job-shop

scheduling. The purpose of this work is to transpose these approaches to the SAFe domain, with its own specificities (WSJF, sprints, specialized teams, *readiness* status).

1.3 Contributions

The thesis proposes six **original** contributions of the SAFESCALE system, each transposing or extending a result published in the literature to the specific context of SAFe PI Planning:

C1.

Six-objective NSGA-II formulation including a WSJF-weighted tardiness (sum of the delays of each feature multiplied by its business value), a transposition to the SAFe context of an objective validated on a multi-project industrial case [15].

C2.

Six initialization seeds including a *WSJF / deadline urgency* ratio adapted from the INSGA-II strategy [1, 2].

C3.

Memetic local search on front 0, inspired by Memetic NSGA-II [4].

C4.

Orchestrated multi-agent architecture in five steps with *conditional* HITL, in line with the multi-agent + NSGA-II architectures [7].

C5.

Critical Chain Buffer per *story*, grounded in Goldratt 1997 and Kingman's law [25], with a deterministic risk score based on 5 weighted factors.

C6.

Complete end-to-end pipeline with a reproducible synthetic dataset calibrated on a real ART.

1.4 Outline of the thesis

Section 2 formalizes the problem and specifies the **scope** of the prototype. Section 3 synthesizes the state of the art in five families. Section 4 describes the model and the key parameters. Section 5 presents the architecture of the flow. Section 6 details NSGA-II. Section 7 synthesizes the design decisions and their sources. Section 8 lists the flow logic problems and the corrections made. Section 9 describes the test protocol and the first results. Section 10 presents the qualitative feedback on the solution. Section 11 sets out the limits and what is not taken into account. Section 12 lists the points to explore. Section 13 concludes. Two appendices (A and B) provide a glossary of acronyms and technical terms.

2 Problem statement

2.1 The existing PI Planning process

Before modeling the problem, one must understand how a PI Planning unfolds *today*, without an optimization tool. The following description comes from an interview with a *Product Owner* (PO) operating on a real ART.

Before the PI Planning. Preparation spans several weeks and mobilizes several roles:

- The **CPO** (*Chief Product Owner*) **prioritizes the features** in his scope and makes sure they can reach

the *ready* state: for this he organizes **co-engineering workshops** with the teams.

- The **duration of epics** is estimated upstream in order to spread the work over the various upcoming PIs.
- The **CPO distributes the features by team**; each team then indicates which ones it is able to address. It sometimes happens that an epic is **split into features distributed across several teams**.
- Once the features are allocated, in **pre-PI Planning**, each PO prepares the scope of his team and lists the topics to handle.

During the PI Planning. The ceremony itself follows a well-established sequence:

- The **Scrum Master** handles facilitation, the **PO** brings the substance of the discussions.
- The **features** retained for the PI are **listed**.
- Each team performs a **poker planning** after the breakdown of features into *stories*, aiming for a size of **8 story points maximum per story**.
- The teams **propose the features** they declare themselves able to address. It is at this stage that the *readiness* status comes into play: if a team identifies **uncertainties** on some items to develop (unclear specifications, unconfirmed external dependency, etc.), it does not *commit* to them for the upcoming PI, and may propose to **push them to the next PI**.

During execution. Planning remains **deliberately general**: at each sprint, the teams hold a **readjustment meeting** to correct the plan according to on-the-ground reality.

Limits of this process. This sequence is mature and proven, but it remains entirely **manual**: the features→teams distribution, the detection of overloads and dependency conflicts, the arbitration between competing priorities all rely on human experience. No **algorithmic exploration** of alternative scenarios is possible, and no **quantitative multi-criteria evaluation** assists the decision. It is precisely this gap that SAFESCALE seeks to fill, by inserting itself as a decision aid **without modifying the existing roles** (CPO, PO, Scrum Master, RTE).

2.2 PI Planning as MS-RCMPSP

Formally, PI Planning is an instance of MS-RCMPSP combining five extensions of the classical RCMPSP [21, 20]:

- *multi-project*: several initiatives share the same resources [17];
- *multi-skill*: resources with several *skills* [16];
- *multi-objective*: six criteria to optimize simultaneously, *makespan*, delay risk, *buffer* deficit, resource overload, capacity violation per team and sprint, WSJF-weighted tardiness, detailed in section 6 [8, 20];
- *team constraints*: *one team owns one feature* [22];
- *inter-feature dependencies* that propagate down to the *stories* [15].

2.3 Five families of recurring conflicts

The interview conducted with a Product Owner, complemented by our modeling, reveals five recurring families of conflicts. Their manual resolution, which mobilizes all the SAFe roles (RTE, CPO, PO, Scrum Master, EDM and dependent teams), is what we seek to instrument:

1. **CAPACITY_VIOLATION** (team beyond its capacity);
2. **DEPENDENCY_BREACH** (*story* before its predecessor);
3. **FEATURE_OVERFLOW** (features that the ART cannot absorb in the current PI and that must be deferred to the next PI);
4. **DEADLINE_RISK** (*feature* past its deadline);
5. **LOAD_IMBALANCE** (>30 % gap between teams).

These five conflicts are *interdependent*: resolving an overload often creates a dependency violation, which creates a *deadline* risk [15].

2.4 Research question

*Can one design a hybrid system combining multi-objective optimization, multi-agent architecture and a risk model, to automate the generation and evaluation of SAFe PI Planning scenarios, while keeping for the human an **effective** decision-making role?*

2.5 Scope of the prototype

The prototype focuses on a precisely defined case study, in order to guarantee the *reproducibility* and legibility of the results:

- **A single ART**, nine specialized teams (*backend, frontend, data, devops, mobile, qa, fullstack* fallback), in line with the 5-15 teams per ART range [22].
- **A PI of four three-week sprints** (60 working days), a format noted during the PO interview and consistent with the 8-12 weeks range [22].
- **About 57 resources** (5-7 per team), 9 epics, ≈ 2 features per epic (≈ 18 features), 5 to 10 *stories* per feature (≈ 135 stories). The *story* is the planning unit: there is no *task* level, in line with the real process observed (section 2.1).
- **Effort $\rightarrow$ duration conversion**: the empirical value varies from one team to another, typically between **1 and 2 working days per story point**. We use **1 SP = 1.7 days** as a pivot value, allowing a per-team draw in the interval [1.0; 2.0].
- **Synthetic data** generated with a fixed *seed* (*seed=42*), guaranteeing full reproducibility of the experiments without access to confidential real data.
- **Batch mode**: a whole PI is planned in a single end-to-end session, triggered by the RTE. “Batch” does not mean “without interaction”: the five steps of the flow include conditional HITL points where the RTE intervenes during execution (cf. section 5). It is the *continuous-flow replanning* between two PIs (for example in reaction to an external event) that is out of scope.

This scope fixes the framework of the work. The elements that fall outside it (real Jira data, simultaneous multi-ART, real-time *replanning*, reinforcement learning) are explicitly deferred to the limits (section 11) and perspectives (section 12).

3 State of the art

The state of the art is organized in five families covering the axes mobilized by the system: multi-objective evolutionary algorithms, extension to more than three objectives, hybrid and multi-project RCPSP, official SAFe practices, and the integration of machine learning into MOEAs.

3.1 Family 1, NSGA-II and its variants

NSGA-II [24] remains, twenty years after its publication, “by far the most popular *metaheuristic* for multi-objective optimization problems” [8]. Three recent axes of improvement structure our contribution:

Axis 1, Targeted initialization. *INSGA-II* [1] demonstrates empirically on 24 instances that a random initialization strongly degrades convergence; the objective-backed strategies (WSJF, deadline-first, critical path, load-balance) significantly improve the optimization rate [1, 2]. Our seed C2 “WSJF / deadline urgency” extends this idea by combining business value and time pressure.

Axis 2, Memetic hybridization. NSGA-II progresses by random “big jumps” (crossovers and mutations): it explores very broadly, but never *refines* a solution. It may pass right next to an optimum without stopping there. The memetic idea consists of adding to NSGA-II a second complementary mechanism, *greedy local search*: one takes the best solutions of the moment and applies to them **small targeted touch-ups** (move a story from one team to another, swap the order of two stories), keeping only those that improve an objective. The table below illustrates the difference:

Without search	local	With (memetic)
One explores broadly at random, but polishes no solution.	explores at random, but polishes	One explores broadly at random, then polishes the best ones with small modifications.
Good coverage, average fine quality.		Good coverage <i>and</i> fine quality.

Memetic NSGA-II [4] applies this recipe to the best individuals of front 0; on job-shop scheduling problems (*flexible job shop*), the authors obtain fronts of higher quality than those of an exact solver, for a much lower computation time. This is the basis of our contribution C3.

Axis 3, NSGA-II + Q-Learning. In classical NSGA-II, the **moment** at which each task starts is chosen by *fixed* rules (“as early as possible once dependencies are satisfied”, for example). The idea of this axis is to replace this frozen rule with a **learned policy**: a *reinforcement learning* agent tries different start dates over many schedules, receives feedback “it was a good or a bad idea” depending on the quality of the final schedule, and progressively *learns* which dates work best. [5] formalizes this as a *Markov decision process*,

solved by the *Q-Learning* algorithm. The table below illustrates the difference:

Without Learning	Q-Learning	With (NSGA-II + Q-Learning)
Start dates chosen by a fixed rule, identical on all schedules.		Start dates chosen by a policy <i>learned</i> on hundreds of previous schedules.
No training needed, but quality limited by the rule.		Costly to train, but better final quality.

The published results surpass pure NSGA-II, PSO and ACO. It is heavy to train but rich in information; we identify it as perspective P3.

What we retain and what is missing: these three axes validate separately targeted initialization, local search and timing learning, but none of these works handles a problem structured like SAFe (epic/feature/story hierarchy, *one team owns one feature* constraint, WSJF). They remain on generic *job shop* test benches; the transposition to PI Planning is not immediate and constitutes precisely a part of our work.

3.2 Family 2, NSGA-III for 5+ objectives

With six objectives, the *crowding distance* of NSGA-II becomes less effective at preserving the uniformity of the front [10]. NSGA-III replaces this mechanism with a system of **reference points** [12, 11], and obtains better fronts than NSGA-II on the RCPSP benchmarks of [12].

What we retain and what is missing: NSGA-III is clearly more suited to our number of objectives, and this is why it appears in perspective P2. But it introduces an additional parameterization, the placement of reference points, which requires a study of its own. Starting with NSGA-II, better documented and simpler to debug, is a deliberate prototype choice, not an end in itself.

3.3 Family 3, hybrid and multi-project RCPSP

The reference *Hybrid Genetic Algorithm* [13] remains a classical basis: it is a standard *genetic algorithm* onto which a *repair-based decoding* is grafted (cf. section 6) to make the *mutation* always valid. [14] introduces the concept of *dense genes*, the genes that “saturate” the use of a resource (close to the critical path). The idea is to use these dense genes as *anchor points* during *crossover*, rather than mixing the chromosomes at random: the good parts of a solution are preserved. On industrial multi-project cases, [15] inspired our 6th objective: the WSJF-weighted tardiness (transposition of the *weighted tardiness* weighted by business criticality). For sustainability, [17] adapts NSGA-II to multi-project, and [16] introduces MOFOA-HS for flexible resource profiles. For duration uncertainty, [18] converts the uncertain (optimistic/pessimistic/likely) into objectives; [19] models costs that vary over time. The review [20] concludes that hybridization generally dominates pure algorithms.

What we retain and what is missing: this family provides the technical building blocks (*decoding*, crossover,

tardiness objective) but reasons in abstract “activities”. It ignores two realities of PI Planning: the breakdown into sprints with fixed capacity, and the role of a human decision-maker who decides at the end of the process. Our work reuses the building blocks but recomposes them around these two constraints.

3.4 Family 4, official SAFe and lean flow

The official SAFe source [22] provides the normative parameters: 5-15 teams per ART, 50-125 people, 8-12 weeks per PI, ROAM board for risks. Reinertsen 2009 [25] formalizes the relationship between utilization and *lead-time* (Kingman’s law): beyond 80 % utilization, the lead-time becomes super-linear. This is the basis of our blocking 80 % buffer (DEC-016) and of the pivot value 2.0 of the `span_factor`.

What we retain and what is missing: SAFe describes *what* to do (the ceremonies, the roles, the artifacts) but not *how* to optimize the content of a PI. The official documentation remains deliberately methodological and offers no quantitative tool. This is precisely the space that this thesis seeks to occupy.

3.5 Family 5, ML + evolutionary algorithms

This family combines **machine learning** (ML) and **multi-objective evolutionary algorithms** (MOEA), the category to which NSGA-II belongs. The common idea: to use a ML model, learned on the history of projects, to steer the optimization toward solutions with a lower predicted *risk*. The *survey* [23] provides an overview. *ENSGA* (*Enhanced NSGA-II, idem*) is an example: it slides a ML model that predicts the fatigue or the risk of operators into the objective function, and optimizes both the total duration (*makespan*) and these human criteria. The multi-agent NSGA-II [7] for its part handles many-objective problems by *distributing* the computation of the *crowding distance* (cf. section 6) among several agents to better cover the front.

What we retain and what is missing: these works show that a risk model can enter the objective function of a MOEA. However, they often mobilize ML learned on large histories. Lacking such a history, we deliberately chose a *deterministic* risk score (equation 7), less ambitious but reproducible and explainable, a deliberate compromise.

3.6 Synthesis of the positioning

Taken separately, these five families each cover an aspect of the problem; none brings them together for SAFe PI Planning. It is this combination that constitutes the contribution of the thesis, summarized in table 1: (i) SAFe-adapted multi-objective NSGA-II with memetics and *urgency* seed, (ii) orchestrated MAS with conditional HITL and trace, (iii) Critical Chain Buffer at the *story* level.

4 Model and data

4.1 Modeled SAFe hierarchy

SAFe distinguishes two axes that must not be confused: an axis of **organization** (who works with whom) and an axis of **work decomposition** (what is broken down into what). Our model reflects both separately:

Dimension	Existing	SAFESCALE
Optim.	mono-obj. RCPSP	6-obj. NSGA-II + LS
MAS	BDI / centralized	Orchestrated + HITL
Risk	Heuristic	Critical Chain + score
Domain	Generic sched.	SAFe (WSJF, sprints)
HITL	Final only	Conditional per step

Table 1: Comparative positioning of SAFESCALE.

organization: $\underbrace{\text{ART} \rightarrow \text{Team} \rightarrow \text{Resource}}$

decomposition: $\underbrace{(\text{Initiative}) \rightarrow \text{Epic} \rightarrow \text{Feature} \rightarrow \text{Story}}$

The *Initiative* level (in parentheses) exists in SAFe above the epic as a large strategic theme; we mention it for completeness but do not model it, for lack of an effect on the planning of a single PI. Our model stops at the *story* on the decomposition axis, the unit actually estimated and planned in the observed process (section 2.1).

To this are added the skills (*Skill*), the dependencies of *features* and *stories*, and the planning artifacts (*PIPlanning*, *StorySchedule*, *Scenario*, *DecisionTrace*). In line with the real process, there is **no task level**: the *story* is the finest grain, estimated in *story points* (SP) during the poker planning, with a ceiling of 8 SP per story.

4.2 Effort, duration and spread conversion

The effort of a *story* is expressed in *story points*. In practice, a story point represents between 1 and 2 effort-days depending on the team; we retain a pivot value of 1.7 days/SP, adjustable per team to reflect this variability:

$$d_s = SP_s \times \alpha_{\text{team}(s)}, \quad \alpha_{\text{team}} \in [1.0; 2.0], \quad \alpha_{\text{default}} = 1.7 \quad (1)$$

But a story estimated at d_s effort-days is never executed over d_s consecutive calendar days: recurring meetings, *context-switching*, technical blockers spread the work out. We model this by a **spread factor**:

$$\text{calendar}(s) = \lfloor d_s \cdot \text{span_factor}_{\text{team}(s)} \rfloor \quad (2)$$

The **span_factor** has a pivot value of 2.0, but it is not identical for all teams: the generator draws for each team a value in the interval $[1.8; 2.2]$. This pivot value of 2.0 corresponds to Kingman's law applied at about 50% utilization (lead-time $\approx 2 \times$ touch time) [25]; the ± 0.2 spread reflects the fact that an experienced team suffers less spread (lower bound) than a junior team (upper bound).

4.3 Team specialization

Each team has a specialization among seven categories (*backend*, *frontend*, *data*, *devops*, *mobile*, *qa*, *fullstack*), in line with the SAFe notion of *Agile Team* [22]. The distribution of skills follows an 80/20 rule: 80% of a resource's skills come from the core specialty of its team, 20% may spill over. Rare skills (*data*, *mobile*, *devops*)

are *exclusive* to the corresponding teams, which should allow an observable capacity bottleneck to be provoked (cf. section 11).

4.4 Reference synthetic dataset

The generator produces, for **seed=42**, a reproducible dataset with dimensions noted on a real ART (section 2.1):

Entity	Quantity
ART	1
Specialized teams (5-15 SAFe)	9
Resources (5-7 / team)	≈ 57
Epics	9
Features (≈ 2 / epic)	≈ 18
Stories (5-10 / feature, ≈ 8 SP)	≈ 135
PI: 4 sprints $\times$ 3 weeks	60 d

5 Architecture and flow

5.1 Technical view in three layers

The application is structured into a **React + TypeScript frontend** (Tailwind, Recharts), a **Python + Django + DRF backend**, and a **PostgreSQL database** (figure 1). A central orchestrator (*discussion_flow.py*) drives specialized agents via a synchronous request/response protocol, in the manner of the multi-agent NSGA-II proposed by [7]. The LLMs intervene exclusively as **translators** (*rationale* in natural language), never as decision-makers; they are *mockable* for the reproducibility of the tests.

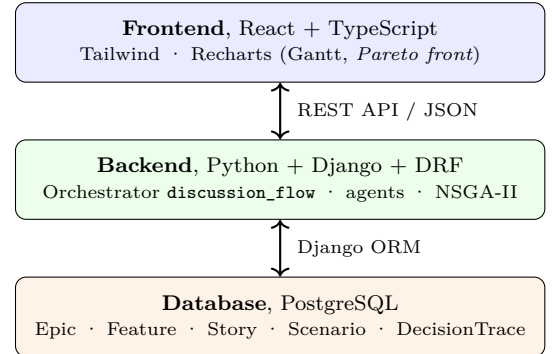


Figure 1: Three-layer architecture of SAFESCALE.

5.2 Five-step flow

The flow is summarized in figure 2:

- Preparation (readiness)**: classification of each feature into READY / BORDERLINE / NOT_READY, in line with the principle of “backlog refinement” [22]. For each team, the *Product Owner* remains the owner of the backlog; he relies on many other actors (CPO, EDM, Scrum Master, dependent teams) who influence the priorities and the state of preparation. HITL if features remain not ready.
- Dependencies**: the directed graph of dependencies between features and between stories (the DAG) is built, then it is traversed **depth-first** (*Depth-First Search*, DFS) for two purposes: (i) detect dependency cycles, which would make planning impossible; (ii) compute the *critical path*, the longest chain

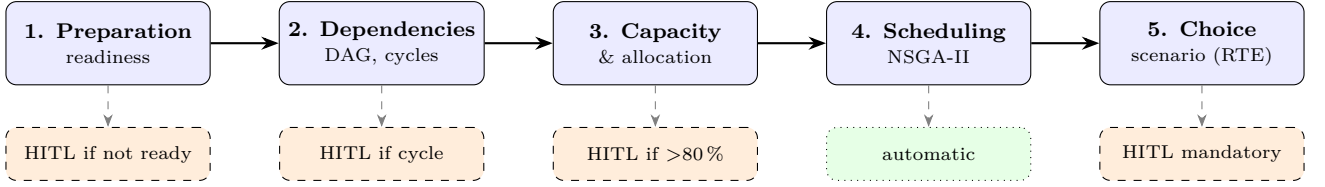


Figure 2: The five-step flow. Step 4 (NSGA-II) is entirely automatic; steps 1, 2 and 3 open a *conditional* HITL point, only if a problem is detected, while step 5 always requires an explicit human decision.

of dependencies that fixes the minimum duration of the PI [15].

3. **Capacity & allocation:** each team builds its capacity plan. **Blocking HITL if a team exceeds 80 % of its capacity** [25].
4. **Scheduling:** NSGA-II (*story* → resource encoding) produces a Pareto front, from which $K = 5$ **diversified** scenarios are extracted. Diversification is evaluated by the *Hamming distance* on the feature→team assignment: two scenarios are all the more different as they assign more features to distinct teams. The *farthest-first* selection (cf. contribution 6) guarantees that each new scenario presented is the farthest possible from the previous ones, which avoids proposing five variants of the same compromise.
5. **Scenario choice: explicit RTE decision**, inspired by the *max-regret* of [5] and the ROAM board of SAFe [22].

5.3 Conditional HITL

Rather than asking at each step “do you want to continue?”, each agent decides locally whether it can decide alone or whether it must open a HITL point. This “conditional” logic is what differentiates SAFESCALE from most existing SAFe tools where the human is either absent, or consulted everywhere.

5.4 Traceability (DecisionTrace)

Each decision is persisted in a `DecisionTrace` model (question asked, decision made, justification, discarded alternatives, *snapshot* of inputs, diff of outputs, human directives, confidence score, LLM *prompt* and response). Multi-level traceability is necessary for the thesis defense supported in the review *Comprehensive Review* on combinatorial NSGA-II [8].

6 NSGA-II: formulation and implementation

6.1 Chromosome encoding

An individual is a list $\mathbf{x} = (x_1, \dots, x_n)$ where $x_i = (\text{story_id}_i, \text{resource_id}_i)$: each gene assigns a *story* to a resource. The order of the genes encodes the execution order. Topological validity is guaranteed at the crossover and mutation by a *repair-based decoding* - a mechanism that, after each genetic operation, traverses the chromosome and automatically reorders the conflicting genes (violated dependency, already busy resource) instead of rejecting the solution, inherited from [13].

6.2 Decoding

$$\text{earliest}_i = \max \{ \text{end}_p \mid p \prec i \} \quad (3)$$

$$\text{start}_i = \max (\text{earliest}_i, \text{avail}_{r(i)}) \quad (4)$$

$$\text{end}_i = \text{start}_i + \text{calendar}(s_i) \quad (5)$$

$$\text{avail}_{r(i)} \leftarrow \text{end}_i + \text{buffer}_i \quad (6)$$

where $\text{calendar}(s_i)$ is the calendar duration of the *story* s_i (equations 1–2) and the buffer buffer_i corresponds to the *Critical Chain Feeding Buffer* defined below.

6.3 Critical Chain Buffer

In line with the Goldratt 1997 principle “remove the individual margins and group them into buffers targeted at the points of variance” (cited in [18]), a deterministic risk score is computed for each *story*:

$$\begin{aligned} \text{risk}(s) = & 0.30 \cdot \tilde{c} + 0.20 \cdot \tilde{\sigma} + 0.20 \cdot \tilde{p} \\ & + 0.20 \cdot \mathcal{K}_{\text{rare}}(s) + 0.10 \cdot \tilde{d} \end{aligned} \quad (7)$$

$$\text{buffer}(s) = \lfloor \text{risk}(s) \cdot 5 \rfloor$$

where $\tilde{c}, \tilde{\sigma}, \tilde{p}, \tilde{d}$ are respectively the complexity, the `span_factor`, the number of predecessors and the effort (in SP), normalized; $\mathcal{K}_{\text{rare}}$ equals 1 if the required skill is *data*, *mobile* or *devops* (underrepresented categories). This modeling extends the idea of [23], which validates empirically the integration of a risk model into the objective function of a MOEA, and that of [18], which converts uncertainty into objectives.

6.4 Six objectives (to minimize)

These six objectives **are not independent**: on the contrary, they contradict one another. If one tries to compress the *makespan*, one parallelizes more, so the resources are overloaded and the risk rises. If, conversely, one keeps margin to lower the risk, one lengthens the makespan and some features may finish after their deadline. There is therefore no single schedule that is the best on the six criteria at the same time. This is exactly why one uses a multi-objective *evolutionary algorithm*: instead of aiming at a single optimum, NSGA-II builds a *Pareto front*, that is, a set of compromises where each gain on a criterion is paid for elsewhere.

Weighting. During the search, NSGA-II **does not weight** the objectives: all are put on an equal footing via the dominance relation (equation 9). The weighting arrives only at the very end, when a “recommended” scenario must be chosen among those of the front. At that moment, a *composite score* is computed (equation 13) whose weights w_j can be adjusted by the RTE. By default, the six weights are each 1/6.

The six objectives are:

1. Makespan $C_{\max}$ [24].
2. Average ML risk $\frac{1}{n} \sum_i risk(s_i)$ [23].
3. Buffer deficit (missing margin).
4. Resource overload (parallelism).
5. Capacity violation per team and sprint [22].
6. **WSJF-weighted tardiness** (contribution C1):

$$f_6 = \frac{1}{n} \sum_i WSJF_i \cdot \max(0, end_i - deadline_i) \quad (8)$$

Transposition to the SAFe context of the *weighted tardiness* objective validated on a multi-project industrial case [15].

6.5 Dominance and Pareto front

Strict Pareto dominance [24]:

$$\mathbf{x} \prec \mathbf{y} \iff \forall j, f_j(\mathbf{x}) \leq f_j(\mathbf{y}) \wedge \exists k, f_k(\mathbf{x}) < f_k(\mathbf{y}) \quad (9)$$

Crowding distance [24]:

$$d_c(i) = \sum_{j=1}^M \frac{f_j(i+1) - f_j(i-1)}{f_j^{\max} - f_j^{\min}} \quad (10)$$

6.6 Six smart seeds

NSGA-II starts with an **initial population** of candidate schedules, typically 40 to 60 individuals. If these schedules are generated completely at random, the algorithm spends the first generations “catching up”. INSGA-II [1, 2] showed that it is much more efficient to **inject a few schedules built by simple heuristics** into the initial population: the evolution then starts near “good regions” of the search space.

Our six generation heuristics each produce a complete initial schedule by ordering the stories according to a business criterion: (1) WSJF descending, business value first; (2) deadline ascending, urgency first; (3) load-balanced, distribute the load fairly; (4) critical path first [15]; (5) minimal ML risk first [23]; (6) **WSJF / deadline urgency ratio** (C2) which combines value and time pressure:

$$urgency(f) = \frac{WSJF_f}{\max(1, deadline_f - now)} \quad (11)$$

The rest of the initial population is completed by random schedules to keep diversity.

6.7 Memetic local search (C3)

Every 5 generations, *greedy* local search on the top 15 % of front 0 (resource reassignment, order swap), following [4]. The mutation is kept only if it degrades no objective (non-domination test).

6.8 Selection of the K=5 diversified scenarios

The Pareto front often contains dozens of non-dominated solutions. One cannot present them all to the RTE, and choosing the five “best” does not work: one quickly ends up with five schedules that all resemble one another, because they are neighbors of the same optimum. The review [8] and the multi-agent NSGA-II [7] insist on the importance of a **good coverage** of the front, not only on the raw quality of the retained solutions.

Measuring the difference between two schedules. We use the *Hamming distance*, which counts the number of positions where two schedules differ. Here, each “position” corresponds to a feature, and the stored value is “which team took it”. Two schedules where all the features are allocated to the same teams have a distance of 0; two schedules that assign each feature to a different team are at maximum distance.

Choosing the 5 most different schedules. We start from the scenario recommended by the composite score, then add one by one the four following ones by applying a rule called *farthest-first*: at each step, we take from the front the one that is the **farthest** from the scenarios already retained. This rule is written:

$$\mathbf{x}_{k+1} = \arg \max_{\mathbf{x} \in \mathcal{F}^0 \setminus \mathcal{S}_k} \min_{\mathbf{y} \in \mathcal{S}_k} d_H(\mathbf{x}, \mathbf{y}) \quad (12)$$

$\mathcal{F}^0$ is the entire Pareto front, $\mathcal{S}_k$ the k scenarios already chosen, and d_H the Hamming distance. At each addition, we guarantee that the new scenario is different from all the previous ones, which avoids the five variants of the same compromise.

Composite score (recommended). To designate the “recommended” scenario by default, each objective is normalized between 0 and 1 and a weighted average is made of them:

$$score(\mathbf{x}) = \sum_{j=1}^6 w_j \cdot \frac{f_j(\mathbf{x}) - f_j^{\min}}{f_j^{\max} - f_j^{\min}} \quad (13)$$

$f_j^{\min}$ and $f_j^{\max}$ are the bounds observed on the front; the w_j are the weights of the six objectives (by default 1/6 each, adjustable by the RTE).

6.9 Removal advice at step 3

When a team exceeds 80 % of load [25], a deterministic impact score is computed:

$$impact(f) = \frac{1}{WSJF_f \cdot (1 + downstream_blocks_f)} \quad (14)$$

The lower the score, the more the *feature* is a candidate for removal (low business value, few cascaded successors). The cascade applies **on_delete=CASCADE** at the Feature→Story level and flags the potentially orphaned DAG successors.

6.10 Capacity bottleneck detection

$$demand(s) = \sum_{t: skill(t)=s} d_t \quad (15)$$

$$supply(s) = \sum_{r: s \in skills(r)} avail_r \cdot PI_{days} \quad (16)$$

$$gap(s) = \max(0, demand(s) - supply(s)) \quad (17)$$

7 Design decisions

Table 2 synthesizes the major design decisions with their justification. Two sources have fed these choices: the recent scientific literature on the one hand, and

on the other a **semi-structured interview with a Product Owner** operating on a SAFe ART. This interview does not have the value of an exhaustive field study, but it made it possible to frame elements that no article documents precisely, the PI format of 4×3 weeks, the empirical story points/days conversion, or the strict application of the 20% buffer. When a decision relies on this interview, the mention *PO Interview* indicates it in the table.

8 Flow logic problems and solutions

The flow described in section 5 was not designed in a single pass. Its first version was more naive, and several logic flaws only appeared when running the system end to end on the reference dataset: a consensus forming on misleading figures, a step traversed without noticing it, an optimizer producing an absurd but “valid” plan. This section gathers these flaws and the correction I made to each. These are not code bugs but errors of **flow design**; I present them under the form *symptom* then *correction*, referring to the corresponding DEC-XXX decision.

8.1 Team agents negotiating on SP aggregates

Symptom. The first version of the flow had the team agents negotiate on estimates aggregated in *story points*. Consequence: their arguments were vague, their portfolio refusals inexplicable, and the swarm could converge on a consensus infeasible on the resource and calendar side, an internal bottleneck (for example a senior resource overloaded at 130% on sprint 2) remained hidden behind the SP average.

Correction (DEC-004). Aligned on the two-phase approach of the Memetic NSGA-II [4]: one first runs a global exploratory pass that produces concrete schedules (*story*→resource assignment, dates, load per sprint), then these schedules are given to the team agents so that they negotiate on real data. A *warm-start* re-scheduling validates the counter-proposals before convergence (DEC-005), in line with the two-stage scheme of [13].

8.2 NSGA-II splitting features across several teams

Symptom. Without an explicit constraint, the optimizer produced technically excellent plans but distributing the *stories* of the same *feature* across 3-4 different teams, a flagrant violation of the SAFe principle “*one team owns one feature*” [22]. The real process admits this breakdown (an epic is sometimes distributed into features across several teams, section 2.1) but treats it as a negotiated exception, never as a default case.

Correction (DEC-006). *Strong soft* penalty ($\times 100$) on each *story* split outside the majority team of its *feature*. A hard constraint would have crashed the solver on legitimate edge cases (*component teams* [22]); the strong soft leaves a valve but eliminates the problem in practice.

8.3 SAFe 20% buffer not respected at step 3

Symptom. The empirical SAFe rule (derived from Kingman’s law [25]: $>80\%$ utilization $\Rightarrow$ super-linear

lead-time) is that one must not exceed 80% of planned load. Yet step 3 let the flow pass to NSGA-II as long as the load remained $\leq 110\%$. NSGA-II then started on an infeasible portfolio and found the “least bad” compromises instead of arbitrating upstream.

Correction (DEC-016). Blocking threshold lowered to 80% by default: as long as at least one team exceeds this threshold or refuses its portfolio, the flow remains in HITL at step 3. The rule is spelled out in a banner citing Reinertsen [25].

Field nuance. The PO interview confirmed that in practice **each team defines its own flow**, and that the percentage of capacity actually dedicatable to PI Planning varies according to the maturity, the topology and the **external load** of the team (support volume, influx of *Problem Change Requests* following an acceptance-testing campaign, etc.). The 80% of the SAFe buffer is therefore a *default target value* and not a universal constant. The blocking threshold is configurable per team in the data model; it is exposed to the RTE so that he can lower this threshold on the teams known to absorb a lot of work outside the PI.

8.4 Overload without a rebalancing loop

Symptom. When step 3 detected an overload, the flow offered **no automatic rebalancing mechanism**: the only possible exit was to continue toward NSGA-II with an infeasible allocation. The algorithm therefore started on an intrinsically oversized portfolio and produced compromises suboptimal by construction.

Correction (DEC-014). Introduction of a **deterministic AI advice** (equation 14) that orders the candidate *features* by increasing impact, and of a **re-CNP loop** (Capacity Negotiation Protocol) bounded by the 80% rule. Following the spirit of the *survey* [23] on ML/MOEA integration in multi-objective decision support, the loop only closes once the capacity constraint is satisfied. The optimizer then receives a healthy portfolio rather than having to compensate downstream.

8.5 Auto-finalization short-circuiting the final decision

Symptom. Step 5 auto-finalized on consensus as soon as the team agents agreed on two preferential strategies or fewer. The *flow* then jumped directly from step 3 to a state “PI Planning finished” without having materialized an explicit choice: the scenario “recommended” by the system was considered accepted by default.

Correction (DEC-015). Step 5 made **always** HITL by construction of the flow. Inspired by the *max-regret* of [5], the orchestrator computes an indicative consensus (majority of votes, recommended by the *composite score*, Pareto-optimal) but never triggers the transition toward step 6 without an explicit decision. This respects the semantics of the conditional HITL: conditional *on an algorithmically detectable problem*, and not on a fundamentally human business decision [22].

Alignment on the real course of a PI Planning. The ritual observed at the interviewee’s takes place over two days with a clear structure, which we seek to reproduce in the tool:

Ref.	Decision	Justification (academic sources + field observation)
DEC-001	Team specialization (enum + <i>full-stack</i> fallback)	[22]: an ART contains 5-15 typed teams. <i>PO Interview</i> : real ARTs are organized into <i>back</i> / <i>front</i> / <i>data</i> / <i>devops</i> / <i>mobile</i> / <i>qa</i> teams with a cross-cutting fullstack team for cross topics.
DEC-002	<code>span_factor</code> pivot 2.0, drawn in [1.8; 2.2] per team	[25] Kingman's law ($\approx 50\%$ utilization $\Rightarrow$ lead-time $\approx 2\times$).
DEC-004	Global exploratory NSGA-II then refinement	[4] (two-phase memetic); [13] (two-stage HGA).
DEC-005	<i>Warm-start</i> re-scheduling of counter-proposals	[13]: two-stage scheme. Validates the <i>feature</i> transfers proposed by the teams before the convergence of the <i>swarm</i> .
DEC-006	feature $\rightarrow$ team cohesion: penalty $\times 100$	[22]: "one team owns one feature"; [16]: flexible profiles with cost. <i>PO Interview</i> : splitting a feature across several teams is treated as a negotiated exception, never as a default case.
DEC-007	Pareto diversity via farthest-first Hamming	[7]: multi-agent crowding; [9]: congestion operators.
DEC-008	Runtime DecisionTrace model	[8]: review insisting on combinatorial traceability.
DEC-010	PI 12 weeks $\times$ 4 sprints of 3 weeks	[22] (PI 8-12 wk.); <i>PO interview</i> : 4×3 weeks format actually used (consistent with the quarterly <i>cadence</i> of Business Owners).
DEC-014	AI advice + HITL rebalancing at step 3	[23]: ML/MOEA integration for decision support; [22]: ROAM board. <i>PO Interview</i> : RTEs perform an explicit <i>capacity review</i> and negotiate removals feature by feature.
DEC-015	Explicit RTE scenario choice at step 5	[5]: max-regret for multi-actor decision; [22]: mandatory Business Owner review. <i>PO Interview</i> : the final decision is never delegated to a mechanical consensus, the RTE decides in the presence of the Product Owners.
DEC-016	Blocking 80 % buffer	[25] (Kingman); [22]. <i>PO Interview</i> : the 20 % buffer rule (capacity load $\leq 80\%$) is strictly applied by RTEs before PI validation.
DEC-018	Critical Chain Buffer per <i>story</i>	Goldratt 1997 (cited in [18]); [23]: ML+MOEA.
C1	WSJF-weighted tardiness (6 th obj.)	[15]: multi-project industrial case, <i>weighted tardiness</i> .
C2	WSJF/deadline urgency seed	[1] INSGA-II; [2] composite rules.
C3	Memetic local search	[4]: <i>greedy</i> local search on front 0.

Table 2: Major design decisions, with their academic sources and, for the choices concerned, the elements from the PO interview.

- **Day 1.** Each PO presents the proposal of his team; the stakeholders (RTE, CPO, EDM, dependent teams) accept or refuse. A *synthesis sharing* at the end of D1 makes it possible to identify the difficulties to anticipate for D2.
- **Day 2.** The teams refine their commitments by integrating the previous day's counter-proposals. A *final validation review* at the end of D2 enacts the definitive plan.
- **Possible outcome.** It happens that the plan *is not accepted* at the end of the two days: in this case, other meetings are held during the first days of the PI itself to bring the teams and the stakeholders toward a validated plan.

SAFESCALE models this structure in two phases on the front-end side: the K scenarios are first presented in *review* mode (analogous to D1, each team votes, the hard points are identified), then the RTE performs the *final validation* (D2) from the votes and the evaluator's advice. The absence of explicit validation at the end of D2 produces a state "PI Planning awaiting arbitration" rather than a state "finished".

8.6 Pareto front giving the illusion of choice

Symptom. The Pareto front contained a few dozen solutions, many of which were neighbors of the same op-

timum. Selecting $K = 5$ by composite score produced scenarios visually different but qualitatively identical.

Correction (DEC-007). We force the **diversification** of the scenarios proposed to the RTE: instead of taking the $K = 5$ best by composite score, we build the list one by one by taking each time the scenario most different from those already chosen. The distance used is the *Hamming distance* on the feature $\rightarrow$ team assignment (cf. section 6 for the detail). Following [7] (front uniformity) and [9] (congestion operators).

8.7 Execution variance and under-activity teams

Field observation. The major constraint observed today on ARTs remains **over-capacity**: teams committed beyond what they can absorb. But the PO feedback also describes the reverse phenomenon — teams **under-activity** on a PI, either because they have a higher velocity than expected, or because their portfolio deflated between planning and execution.

Symptom on the optimizer side. With equation 2 alone, two problems appear:

- a team planned at 80 % sees its schedule compact over 2 sprints whereas it has 4, without any margin for execution variance;
- the optimizer proposes nothing to make use of the

capacity freed up by an under-activity team.

Correction (DEC-018) — for execution variance. Insertion of **Critical Chain buffers** targeted at the high-variance *stories* (equation 7), in line with Goldratt 1997 and the ML+MOEA *survey* [23]. A variant of the *fuzzy RCPSP* [18] that converts uncertainty into objectives; here we convert it into local buffers.

Lead for under-activity (to explore). When a team is under-utilized, two business levers are conceivable, in addition to the buffers:

- **Anticipate stories from the next PI:** pull up in the backlog features planned for PI N+1 and add them opportunistically if the skill fits;
- **Take on stories from other teams** that are overloaded, provided the specialization allows it (*cross-team support*, typically by a *fullstack* team).

These two movements are already partially modelable in SAFESCALE: the first fits into the feature removal/addition mechanism of step 3, the second depends on the specialization model already described in section 4. Their explicit integration is listed as perspective P7 (section 12).

8.8 Uninstrumented per-skill capacity bottleneck

Symptom. A bottleneck on a rare skill (*ml_ops*, *mobile*, *devops*) never stands out as an isolated signal: the overload manifests at the team scale and drowns the origin. The RTE removes *features* “at random” without knowing which one unblocks the bottleneck.

Correction. A *skill_bottleneck* service that computes *demand*, *supply* and *gap per skill*, identifies the covering teams, and weights the candidate *features* for removal by the effort they free up on the bottleneck skill. Extreme case: a skill not covered by any team $\Rightarrow$ bonus +5 to the removal score of the *features* that require it. This modeling relies on the notion of *flexible resource profile* of [16].

9 Tests and experimental results

This section will be completed at the end of the experimental campaigns. It will present the retained protocol, the metrics of Pareto front quality (*hypervolume*, *spread*), the comparison between standard NSGA-II and its variants (seeds, memetic), the behavior of the flow step by step, as well as the results of the automated tests (success rate, coverage).

Two additional protocols are planned:

- **Agent vs human PO comparison.** On the same dataset, a PO is asked to build a PI Planning by hand, then SAFESCALE is launched. We measure the gaps in *features* $\rightarrow$ teams allocation, in execution order and in KPI; the divergences reveal either problems of criteria weighting in the system, or arbitrations that a human would not have seen.
- **Scenario labeling.** The K scenarios of the selected front (*cf.* section 10) will be automatically named according to deterministic rules based on the dominant KPIs (for example “minimal makespan”, “lowest risk”, “best-balanced load”), so that the labels speak directly to the PO without knowledge of the underlying algorithms.

10 Feedback on the solution

10.1 Qualitative observations

The feedback that follows is **qualitative**: it comes from demonstration sessions conducted internally on the reference dataset, and not from a quantified evaluation, which will be the subject of section 9 once the campaigns are finished.

Flow legibility. Breaking the process into five named steps (*Preparation*, *Dependencies*, *Capacity & allocation*, *Scheduling*, *Scenario choice*) makes each arbitration easier to follow. One can resume a session step by step and explain what happened, which is useful for the defense.

Effect of the 80 % buffer. As long as the blocking threshold did not exist, step 3 was traversed quickly and without real discussion. By making it blocking, it becomes a moment of negotiation: the RTE sees explicitly why he must remove *features*, since the rule is displayed.

Removal advice. The removal panel at step 3 orders the candidate *features* by impact score (equation 14). In practice, it highlights unintuitive arbitrations, for example that a *feature* with average WSJF without a *deadline* frees up more margin than a *feature* with high WSJF but a tight *deadline*.

10.2 Pareto consistency

The *farthest-first* selection (DEC-007) produces visibly distinct scenarios: one can give them speaking labels (“fast”, “safe”, “lean”). This is the intended effect, avoiding presenting to the RTE five variants of the same compromise.

How these labels are generated. Once the five scenarios are selected, we look at which of the six objectives each is the **best of the lot** on and derive its name:

Best on...	Generated label
makespan	“fast”
ML risk	“safe”
capacity violation	“lean”
WSJF tardiness	“priority”
buffer deficit	“with margin”
overload	“balanced”

The names are deliberately **short and business-oriented**: the PO and the RTE can use them to talk to each other (“I prefer the fast scenario, but with a bit more margin”) without knowing the mathematical formula that determined the ranking. If several scenarios are best on the same objective, we decide by the second best KPI, which produces compound labels (“fast-safe”).

10.3 Reproducibility

The fixed *seed*, the LLM *mock* mode and the *DecisionTrace* trace make it possible to replay a session identically. This is important here: without it, one could neither compare two versions of the system, nor verify a result during the defense.

11 Limits and what is not taken into account

The scope of the prototype was specified in section 2.5. This section lists what was deliberately excluded from it, the known algorithmic limits, and the experimental limits of the reference dataset.

11.1 Functional out-of-scope

- **Real Jira data** not yet ingested: the move to production requires adaptation to the Jira Agile@Scale format as well as authorization management.
- **Multi-ART** not supported: planning is done on one ART at a time. Multi-project exists (epic → feature) but not simultaneous multi-ART, addressed in the literature [17].
- **Real-time replanning** not handled: the system operates in *batch* mode, continuous *replanning* during a PI belongs to another class of problem.
- **Reinforcement learning** not implemented: the NSGA-II + Q-Learning hybridization of [5] remains to be explored.

11.2 Algorithmic limits

- **NSGA-II with crowding distance**: NSGA-III [10, 12] would be more suited for 6 objectives.
- **No global end-of-PI buffer**. As it stands, we insert a small buffer *after each risky story* to absorb its possible delays (this is what is called a *feeding buffer*). Goldratt's method additionally recommends a **single large buffer at the very end of the PI** (the *project buffer*), which serves as an aggregated safety net for the whole ART. We have not yet put it in place: the RTE therefore drives dozens of small local buffers instead of a single global buffer, which makes it more difficult to see whether the PI as a whole is in danger.
- **span_factor = 2 is a lower bound**: the Kingman law gives a lead-time that grows hyperbolically beyond 80 % utilization [25]; on high-variance projects, the coefficient underestimates the real lead-time.

11.3 Experimental limits

On `seed=42` the capacity *gap* remains negative on all skills, so the HITL scenario of the AI removal on a *data* bottleneck does not trigger spontaneously. A `--amplify-bottleneck` mode or an enrichment of the templates with more *data features* is necessary to make the demonstration clinically triggerable. More generally, the dataset is synthetic: external validity is limited and will require a replication on real data.

12 Points to explore

Beyond the delivered version, six extension leads are identified. They are deliberately technical, each addresses a precise point of the algorithm or the system, but we announce them here without going into the implementation details. The reader interested in the underlying mechanisms can refer to section 3 (state of the art).

12.1 P2: Move to NSGA-III to better handle 6 objectives

In one sentence: replace NSGA-II with a more recent variant, NSGA-III, better equipped to compare sched-

ules on many criteria at the same time.

NSGA-II uses the *crowding distance* to preserve the diversity of the Pareto front. This technique is effective up to three objectives, but loses precision beyond. NSGA-III replaces this mechanism with a system of **reference points** placed on a normalized hyperplane, which explicitly guides the coverage of the front [10, 12, 11]. At six objectives as we have, the gain is documented in the literature [7]; the cost is an additional parameterization (placement of the points), which requires its own study.

12.2 P3: NSGA-II + Q-Learning

In one sentence: let the system *learn* the good start dates, instead of choosing them by a fixed rule.

[5] demonstrates that learning a *timing* policy (choice of the `start_day` of each activity) as a Markov decision process improves the results relative to pure NSGA-II, PSO and ACO. It is the natural complement to our *urgency* seed which currently rests on a fixed heuristic.

12.3 P4: Real Jira data and multi-ART

In one sentence: plug SAFESCALE onto the real Jira data and let it plan several ARTs at a time.

[17] adapts NSGA-II to multi-project with sustainability criteria, filling a gap in the literature. Combined with a Jira Agile@Scale adapter, this work would allow the move from the prototype to an industrial framework.

12.4 P5: Global project buffer

In one sentence: replace the dozens of small local buffers with a single large global end-of-PI buffer, easier to drive.

Consolidating the local *feeding buffers* of DEC-018 into a global end-of-PI *project buffer* in Goldratt's 1997 sense (cited in [18]) would allow the RTE to drive a single aggregated buffer rather than dozens of local buffers. The algorithmic cost is low but the change of mental model on the user side requires a study.

12.5 P6: Flexible resource profiles

In one sentence: let the system adjust on the fly how many people work on a story, rather than freezing it at planning time.

[16] introduces MOFOA-HS for flexible profiles where the amount of resource per activity is determined in real time. Relevant for resources shared between SAFe teams via the *component teams*.

12.6 P7: Make use of under-activity teams

In one sentence: when a team has free capacity, propose that it anticipate stories from the next PI or come to the aid of an overloaded team.

The field feedback reveals a symmetric case of over-capacity: teams committed below their potential on a given PI. Two levers are to be instrumented in the tool: (i) propose to the RTE to **anticipate stories from the next PI** compatible with the available skills of the under-utilized team; (ii) allow an under-activity team to **take on stories from other overloaded teams**, subject to specialization compatibility. The

skill modeling (cf. section 4) and the removal/addition mechanism at step 3 already lay the necessary building blocks; it remains to wire the decision rules and the RTE interface for these two movements.

13 Conclusion

This thesis presents SAFESCALE, a functional prototype for SAFe PI Planning combining six-objective NSGA-II, an orchestrated multi-agent architecture and a Critical Chain risk model. The main contribution is not a new algorithm but the assembly of already-proven elements ([4, 1, 23, 25]) in the precise context of SAFe PI Planning, WSJF, sprints, specialized teams, *readiness* status, inter-feature dependencies, a context that the interview with a Product Owner helped to frame.

The decisions table (table 2) links each design choice to its source, an article or a point noted during the PO interview. It is a means, more than a stylistic effect, to be able to justify the system before a jury.

At this stage, the work remains a prototype: the flow runs end to end on the reference dataset, but the quantified evaluation is not yet done (section 9). The qualitative feedback suggests that the breakdown into five steps with conditional HITL makes the arbitrations legible; this will have to be confirmed by measurements. The limits (section 11) and the extension leads (section 12) indicate the next steps toward a real use.

A Appendix: Glossary of acronyms

ACO

Ant Colony Optimization, optimization by ant colonies.

ART

Agile Release Train, grouping of 5 to 12 teams that deliver together.

BDI

Beliefs, Desires, Intentions, autonomous agent model (not retained here).

CNP

Capacity Negotiation Protocol, step 3 of the flow, capacity negotiation.

CPO

Chief Product Owner, prioritizes the features of his scope.

DAG

Directed Acyclic Graph, directed graph without a cycle, models the dependencies.

HITL

Human-In-The-Loop, intervention of a human in the decision loop.

LLM

Large Language Model, used here as a translator.

MAS

Multi-Agent System.

ML

Machine Learning.

MOEA

Multi-Objective Evolutionary Algorithm.

MS-RCMPSP

Multi-Skill Resource-Constrained Multi-Project Scheduling Problem, multi-skill and multi-project extension of the RCMPSP.

NSGA-II/III

Non-dominated Sorting Genetic Algorithm - non-dominated sorting genetic algorithm, versions 2 and 3.

PI

Program Increment, delivery cycle of 8 to 12 weeks.

PO

Product Owner, carries the content of a team.

RCPSP

Resource-Constrained Project Scheduling Problem - project scheduling problem under resource constraints.

ROAM

Resolved, Owned, Accepted, Mitigated, SAFe risk management board.

RTE

Release Train Engineer, facilitator of the ART.

SAFe

Scaled Agile Framework.

SP

Story Points, relative unit of effort estimation.

WSJF

Weighted Shortest Job First, prioritization score: value over effort.

B Appendix: Glossary of technical terms

Optimization and evolutionary algorithms

Evolutionary algorithm

Algorithm that imitates natural evolution (crossover, mutation, selection) on a population of solutions.

Metaheuristic

General method that finds very good solutions without guaranteeing the exact optimum.

Individual / chromosome

A candidate solution, here, a complete schedule.

Gene

Smallest element of a chromosome; here, the pair (*story*, resource).

Crossover

Combination of two parent solutions to produce a child.

Mutation

Random modification of a part of a solution.

Elitism

Systematic preservation of the best solutions from one generation to the next.

Pareto front

Set of the best compromises: no solution of the front can be improved on one criterion without degrading another.

Dominance

A dominates B if A is at least as good everywhere and strictly better somewhere.

Crowding distance

Measure favoring well-distributed solutions on the front.

Local search

Step-by-step improvement of a solution by small modifications.

Memetic algorithm

Evolutionary algorithm augmented with a local search.

Seed (initialization)

Smart starting solution injected into the initial population.

Seed (random seed)

Number fixing the randomness; at a fixed seed, the program is reproducible.

Hamming distance

Number of positions where two solutions differ.

NP-hard

Problem for which no known algorithm gives the optimum in reasonable time at large size.

Scheduling

Makespan

Total duration of the project, from the start of the first story to the end of the last.

Critical path

Longest chain of dependencies; fixes the minimum duration of the project.

Topological sort

Ordering of a DAG respecting all the dependencies.

Decoding

Transformation of a chromosome into a concrete schedule (start/end dates).

Weighted tardiness

Delay of a delivery multiplied by the importance of the delayed element.

Bottleneck

Skill too much in demand relative to the number of people who master it.

SAFe and agility

Epic / Feature / Story

Levels of work breakdown; the *story* is the planning unit.

Story point

Relative unit of effort estimation of a story.

Velocity

Story points completed per team and per sprint.

Poker planning

Collective estimation of the effort of the stories.

Readiness

Level of preparation of a feature before planning.

Sprint / cadence

Short iteration (3 weeks); the cadence is the regular rhythm of the ceremonies.

Component team

Cross-cutting team; exception to the principle “one team = one feature”.

Architecture and modeling

Orchestration / emergence

Orchestration: a central conductor drives the agents. Emergence: autonomous agents without a leader.

Conditional HITL

The human is consulted only if a problem is detected.

Mock

Fake version of a component for reproducible tests.

Effort-duration / calendar-duration

The real effort (person-days) against the time actually elapsed (lead-time).

Span factor

Coefficient transforming the effort-duration into calendar-duration (pivot 2.0).

Kingman's law

Beyond ~80 % utilization, the waiting delays explode, justifies the 20 % buffer.

Critical Chain

Goldratt's method: group the margins into targeted buffers rather than inflate each estimate.

Feeding buffer

Time buffer inserted after a risky story.

Deterministic risk score

Score 0-1 computed by a fixed formula (same inputs $\Rightarrow$ same score).

Evaluation

Synthetic dataset

Artificially generated dataset, with dimensions calibrated on a real case.

Reproducibility

Property of an experiment that gives back the same result at each execution.

Hypervolume

Volume of the objective space covered by a Pareto front; larger = better.

Spread

Measure of the good distribution of the solutions on the front.

Composite score

Single mark combining the six objectives normalized and weighted.

Technical debt

Known corrective work, deliberately deferred and documented.

References

- [1] X. Wen, Q. Song, Y. Qian *et al.*, “Effective Improved NSGA-II Algorithm for Multi-Objective IPPS”, *Mathematics*, MDPI, 11(16):3523, 2023.
<https://ideas.repec.org/a/gam/jmathe/v11y2023i16p3523-d1217570.html>
- [2] “An Improved NSGA-II Method for Solving Multi-objective Flexible Job-shop Scheduling Problems”, *ResearchGate*, 2024.
<https://www.researchgate.net/publication/388139247>
- [3] J. Wang, L. Xu, S. Sun *et al.*, “Multi-objective optimization using improved NSGA-II for IPPS in a machining job shop for large-size valve”, *PLoS ONE*, 19(6):e0306024, 2024.
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0306024>
- [4] “A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs”, *Flexible Services and Manufacturing Journal*, Springer, 2024.
<https://link.springer.com/article/10.1007/s10696-023-09517-7>
- [5] “Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA-II and Q-learning”, *ScienceDirect*, 2024.
<https://www.sciencedirect.com/science/article/abs/pii/S156849462301219X>
- [6] “Is NSGA-II Ready for Large-Scale Multi-Objective Optimization?”, MDPI, 2022.
<https://www.mdpi.com/2297-8747/27/6/103>
- [7] “Multi-objective optimization method for building energy-efficient design based on multi-agent-assisted NSGA-II”, *Energy Informatics*, Springer, 2024.
<https://link.springer.com/article/10.1186/s42162-024-00394-4>
- [8] S. Verma, M. Pant, “A Comprehensive Review on NSGA-II for Multi-Objective Combinatorial Optimization Problems”, *IEEE Xplore*, 2021.
<https://ieeexplore.ieee.org/document/9393947/>
- [9] Wang Yahui, Shi Ling, Zhang Cai *et al.*, “NSGA-II algorithm and application for multi-objective flexible workshop scheduling”, *Sage Journals*, 2020.
<https://journals.sagepub.com/doi/full/10.1177/1748302620942467>
- [10] K. Deb, H. Jain, “An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I”, *IEEE TEVC*, 18(4):577-601, 2014.
<https://ieeexplore.ieee.org/document/6600851>
- [11] “Research on Multi-Objective Optimization of Project Scheduling Based on NSGA-III”, *International Core Journal of Engineering*, 11(4), 2025.
<https://bcpublication.org/index.php/ICJE/article/view/8360>
- [12] V. H. S. Pham, N. T. P. Hoang, “Advancing multi-objective optimization in resource management using the NSGA-III algorithm”, *Canadian Journal of Civil Engineering*, 52(7): 1319-1335, 2025.
<https://cdnsiencepub.com/doi/10.1139/cjce-2024-0298>
- [13] “A hybrid genetic algorithm for the resource-constrained project scheduling problem”, *EJOR*, 185(2), 2008.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221707000616>
- [14] E. Goncharov, “A hybrid heuristic algorithm for the resource-constrained project scheduling problem”, *arXiv:2502.18330*, 2025.
<https://arxiv.org/abs/2502.18330>
- [15] “Solving a real-life multi-skill resource-constrained multi-project scheduling problem”, *Annals of Operations Research*, Springer, 2024.
<https://link.springer.com/article/10.1007/s10479-023-05784-7>
- [16] “Multi-Objective Multi-Skill Resource-Constrained Project Scheduling Considering Flexible Resource Profiles”, MDPI *Applied Sciences*, 14(5), 2024.
<https://www.mdpi.com/2076-3417/14/5/1921>
- [17] Lostumbo, Bara, Iniasta, Calvet, “A NSGA-II Algorithm for Resource-Constrained Multi-Project Scheduling Problem Considering Sustainability Criteria”, *SSRN*, 2024.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5060061

- [18] “Fuzzy Resource Constrained Project Scheduling Optimization with Hybrid Multiobjective Genetic Algorithm”, ACM ICBIC, 2021.
<https://dl.acm.org/doi/10.1145/3448748.3448793>
- [19] “Multi-mode RCPSP with time-dependent resource costs and capacities: A bi-objective approach”, *ScienceDirect*, 2025.
<https://www.sciencedirect.com/science/article/pii/S0957417425045701>
- [20] R. Pellerin *et al.*, “A survey of hybrid metaheuristics for the resource-constrained project scheduling problem”, *EJOR*, 280(2), 2020.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221719300475>
- [21] W. Herroelen, B. De Reyck, E. Demeulemeester, “Resource-constrained project scheduling: A survey of recent developments”, *Computers & Operations Research*, 25(4), 1998.
<https://www.sciencedirect.com/science/article/abs/pii/S0305054897000551>
- [22] Scaled Agile Inc., *SAFe 6.0 Framework, PI Planning*.
<https://framework.scaledagile.com/pi-planning>
- [23] “Enhancing multi-objective evolutionary algorithms with machine learning for scheduling problems: recent advances and survey”, *Frontiers in Industrial Engineering*, 2024.
<https://www.frontiersin.org/journals/industrial-engineering/articles/10.3389/fieng.2024.1337174/full>
- [24] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”, *IEEE Transactions on Evolutionary Computation*, 6(2):182-197, 2002.
<https://ieeexplore.ieee.org/document/996017>
- [25] D. G. Reinertsen, *The Principles of Product Development Flow: Second Generation Lean Product Development*, Celeritas Publishing, 2009.
https://www.researchgate.net/publication/235268824_The_Principles_of_Product_Development_Flow
- [26] J. Blazewicz, J. K. Lenstra, A. H. G. Rinnooy Kan, “Scheduling subject to resource constraints: classification and complexity”, *Discrete Applied Mathematics*, 5:11-24, 1983.
<https://www.sciencedirect.com/science/article/pii/0166218X83900124>